

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

FACULTAD DE CIENCIAS DE LA INGENIERÍA

[Nombre de la Carrera]

PROYECTO FIN DE CURSO

ENTREGA 1B

Módulo de Autenticación JWT + Acceso a Datos con Spring Data JPA

Título del PFC: ARTISYN

Asignatura: *[Nombre de la Asignatura]*

Docente: *[Nombre del Docente]*

Entrega: Entrega 1B — Sumativa #8

Fecha de entrega: Semana 9 — Del 05 al 11 de Enero del 2026

Rama Git: entrega-1b

URL del repositorio: <https://github.com/JohanCarvajal04/Proyecto-WEB-ARTISYNC.git>

Integrantes del Equipo

Nombre Completo	Cédula	Rol en el Proyecto
Carvajal Loor Johan Stalin	1207452903	Backend Auth / Líder Técnico

Figueroa Morales Bryan Javier	1206395202	Backend CRUD / Datos
Rios Cuyabazo Jhon Kevin	1251038236	Backend Integración / Infra + Inform

Stack: Java 25 · Spring Boot 4.1.0 · Angular 22 · PostgreSQL 18 · Redis 7 · Docker Compose

1. Resumen Ejecutivo

En la presente entrega se implementó el módulo de autenticación stateless basado en JSON Web Tokens (JWT) y el módulo de acceso a datos para el sistema **ARTISYNC**, una plataforma web orientada a la economía creativa que busca conectar a creadores de contenido (artistas, músicos, programadores, diseñadores) con clientes interesados en sus servicios y productos digitales. El proyecto aborda la problemática de la fragmentación de herramientas para estos profesionales, centralizando la gestión de perfiles, catálogo, pedidos y comunicación. Se utilizó un stack tecnológico moderno con Java 25 / Spring Boot 4.1.0 para el backend, Angular 22 para el frontend, PostgreSQL 18 como motor de base de datos, Redis 7 para la gestión de la lista negra de tokens y Docker Compose para la orquestación de todos los servicios. Como resultados concretos, se logró un sistema de autenticación funcional que emite tokens JWT con un tiempo promedio de **180 ms**, un CRUD completo para la entidad **Servicio** con paginación, y una suite de pruebas JUnit 5 con 10 casos, todos en estado PASS, logrando una cobertura del **82%** en el módulo de seguridad. La integración con Redis permitió implementar una lista negra de JTI con TTL dinámico, garantizando la invalidación inmediata de tokens revocados y mejorando la postura de seguridad del sistema. El despliegue con `docker compose up` es reproducible y funcional, demostrando la viabilidad de la arquitectura propuesta en la Entrega 1A.

2. Introducción

2.1 Contexto y Motivación

En la actualidad, la economía creativa y de libre dedicación (freelance) en el Ecuador carece de ecosistemas digitales unificados que centralicen de punta a punta la cadena de valor de los profesionales independientes. Artistas, músicos, programadores y diseñadores gráficos se ven obligados a fragmentar sus operaciones utilizando un promedio de tres a cuatro plataformas simultáneas para promocionar su portafolio, gestionar los requerimientos de los clientes, mitigar disputas y procesar recaudaciones económicas. Esta dispersión no solo incrementa la complejidad administrativa y los costos transaccionales, sino que expone a los trabajadores a altos índices de desprotección e informalidad financiera.

La problemática se agudiza por la ausencia de mecanismos locales automatizados para la validación de identidad y la falta de garantías en los flujos de pago por encargo. Ante este escenario, ARTISYNC nace como una solución arquitectónica integral que mitiga estos riesgos mediante la centralización absoluta del proceso de contratación y venta dentro de un entorno web seguro. Al integrar componentes automatizados de verificación de identidad, un motor dinámico de flujos de trabajo basados en contratos digitales y un patrón de retención de fondos (Escrow) bajo pasarelas de pago globales, el sistema suprime la necesidad de recurrir a canales de comunicación o liquidación externos, resguardando la integridad operativa y la propiedad intelectual de la comunidad creativa.

2.2 Objetivos de la Entrega 1B

1. Implementar el módulo de autenticación stateless y el control de acceso basado en roles (RBAC) empleando Spring Security 6 y la biblioteca especializada jjwt 0.12, garantizando la seguridad en la capa de servicios de la API REST.
2. Configurar de forma persistente la gestión de tokens de refresco y la lista de revocación inmediata (Blacklist) de identificadores únicos de token (JTI) sobre un almacén de datos, mitigando riesgos de secuestro de sesión.
3. Desarrollar el ciclo de vida transaccional completo (CRUD) para la entidad principal Servicio utilizando las abstracciones de repositorio de Spring Data JPA, Hibernate 6 y el versionamiento estructurado de base de datos mediante Flyway.
4. Demostrar la portabilidad y el despliegue automatizado y reproducible de la infraestructura completa (backend, frontend, base de datos y caché) por medio de la orquestación de contenedores con Docker Compose.

2.3 Alcance

- **Módulos implementados en esta entrega:** Autenticación JWT (registro, login, logout, refresh), CRUD de la entidad Servicio (crear, listar paginado, obtener por ID, actualizar, eliminar lógico), frontend Angular con interceptor JWT y guard de rutas básico, y despliegue con Docker Compose.
- **Módulos fuera de alcance (Entrega 2):** Implementación del flujo de pedidos, sistema de mensajería en tiempo real (WebSockets), integración con pasarela de pagos (PayPal Escrow), y módulos de perfil de

creador y portafolio.

2.4 Estructura del Documento

El presente informe se organiza en las siguientes secciones:

1. Resumen Ejecutivo: síntesis ejecutiva con métricas clave.
2. Introducción: contexto, objetivos y alcance.
3. Marco Teórico: fundamentos académicos de las tecnologías empleadas.
4. Diseño del Sistema: diagramas C4, clases UML, secuencia JWT y DER.
5. Implementación: estructura del repositorio, configuración y código relevante.
6. Pruebas: casos JUnit 5, evidencias Postman y métricas de rendimiento.
7. Seguridad Implementada: controles OWASP documentados.
8. Docker Compose y Despliegue: configuración y README.
9. Historial de Git: commits y convenciones.
10. Conclusiones: evaluación de objetivos y proyección a Entrega 2.
11. Referencias Bibliográficas

3. Marco Teórico

3.1 Autenticación Stateless vs. Stateful y JWT

La autenticación es el proceso de verificar la identidad de un usuario. Tradicionalmente, los sistemas web utilizan un enfoque *stateful* basado en sesiones, donde el servidor mantiene el estado de la sesión del usuario (ej. en memoria o en una base de datos) y la identifica mediante una cookie de sesión. Esto puede ser un cuello de botella en sistemas con muchos usuarios y dificulta la escalabilidad horizontal. Por el contrario, un enfoque *stateless* no almacena ningún estado en el servidor. En su lugar, toda la información necesaria para autenticar al usuario se incluye en un token, que el cliente envía en cada solicitud. JSON Web Token (JWT) (Jones et al., 2015) es un estándar abierto (RFC 7519) para crear tokens de acceso *stateless*. Está compuesto por tres partes codificadas en Base64: un *header* con el algoritmo de firma, un *payload* con los *claims* (datos del usuario como *sub*, *email*, *rol*), y una *firma* que garantiza la integridad del token. Su uso en arquitecturas SPA/API REST es ampliamente recomendado por su simplicidad y escalabilidad, y su seguridad se basa en la correcta gestión de la clave secreta y el tiempo de expiración (Spilca, 2024).

3.2 Spring Security 6 y el Modelo de Filtros

Spring Security es el estándar *de facto* para la seguridad en aplicaciones Spring. Su arquitectura se basa en una cadena de filtros (*SecurityFilterChain*), donde cada filtro intercepta las solicitudes HTTP para realizar una tarea de seguridad específica, como la autenticación o la autorización. El *AuthenticationManager* es el componente central que delega la verificación de credenciales a uno o varios *AuthenticationProviders*, siendo el más común el *DaoAuthenticationProvider* que utiliza un *UserDetailsService* para cargar los datos del usuario desde una fuente de datos (como una base de datos). En esta entrega, se extiende la cadena de filtros con un *OncePerRequestFilter* personalizado (*JwtAuthFilter*) que extrae el token JWT de la cabecera *Authorization: Bearer*, lo valida y establece la autenticación en el *SecurityContextHolder*, permitiendo que el resto de la aplicación conozca al usuario autenticado.

3.3 ORM y Mapeo Objeto-Relacional con Hibernate/JPA

El mapeo objeto-relacional (ORM) es una técnica que permite a los desarrolladores interactuar con una base de datos relacional utilizando un paradigma de objetos, sin necesidad de escribir SQL directamente. Hibernate, la implementación más popular

de la especificación Java Persistence API (JPA), actúa como un "puente" entre el modelo de objetos de la aplicación y el modelo relacional de la base de datos. El uso de Hibernate/JPA (Walls, 2022) frente a JDBC puro aporta ventajas significativas como la reducción de código *boilerplate*, el manejo automático de relaciones entre tablas, la portabilidad entre diferentes motores de base de datos, y la optimización mediante caché de primer nivel y generación de consultas derivadas del nombre de los métodos. En este proyecto, se utiliza Spring Data JPA, que simplifica aún más la implementación de repositorios mediante la extensión de interfaces como `JpaRepository`.

3.4 Migraciones de Base de Datos con Flyway

En un entorno de desarrollo colaborativo, el versionado del esquema de la base de datos es fundamental para garantizar la consistencia entre los diferentes entornos (desarrollo, pruebas, producción). Flyway es una herramienta de migración de bases de datos que permite aplicar cambios al esquema de manera controlada y repetible. Los cambios se definen como scripts SQL versionados, por ejemplo, `V1_schema_inicial.sql`. Al ejecutarse, Flyway lleva un registro de las migraciones aplicadas en una tabla de metadatos, asegurando que cada script se ejecute solo una vez y en el orden correcto. En este proyecto, se utiliza `ddl-auto: validate` en Spring Boot para verificar que el modelo de entidades JPA coincida con el estado del esquema gestionado por Flyway, evitando la generación de DDL automática que podría llevar a inconsistencias (PostgreSQL Global Development Group, 2024).

3.5 OWASP Top 10 y Seguridad JWT

El OWASP Top 10 (OWASP Foundation, 2021) es un documento de concienciación sobre los riesgos de seguridad más críticos en aplicaciones web. La implementación de JWT en esta entrega aborda varios de estos riesgos. La falla A07 (Fallas de Autenticación) se mitiga mediante el uso de tokens de corta duración, el almacenamiento seguro de contraseñas con BCrypt y la implementación de una lista negra para tokens revocados. La A02 (Fallas Criptográficas) se aborda utilizando el algoritmo de firma HS256 y una clave secreta de 256 bits. La A03 (Inyección) se previene mediante el uso de Spring Data JPA y consultas parametrizadas (`@Query` con `?` o `:nombreParam`), evitando la concatenación de SQL. La A05 (Mala Configuración de Seguridad) se trata configurando cabeceras HTTP de seguridad (X-Content-Type-Options, X-Frame-Options) y una política CORS estricta. La A01 (Control de Acceso Roto) se mitiga mediante el uso de `@PreAuthorize` para restringir el acceso a endpoints según los roles de usuario.

4. Diseño del Sistema

4.1 Diagrama C4 Nivel 2 (Contenedores)

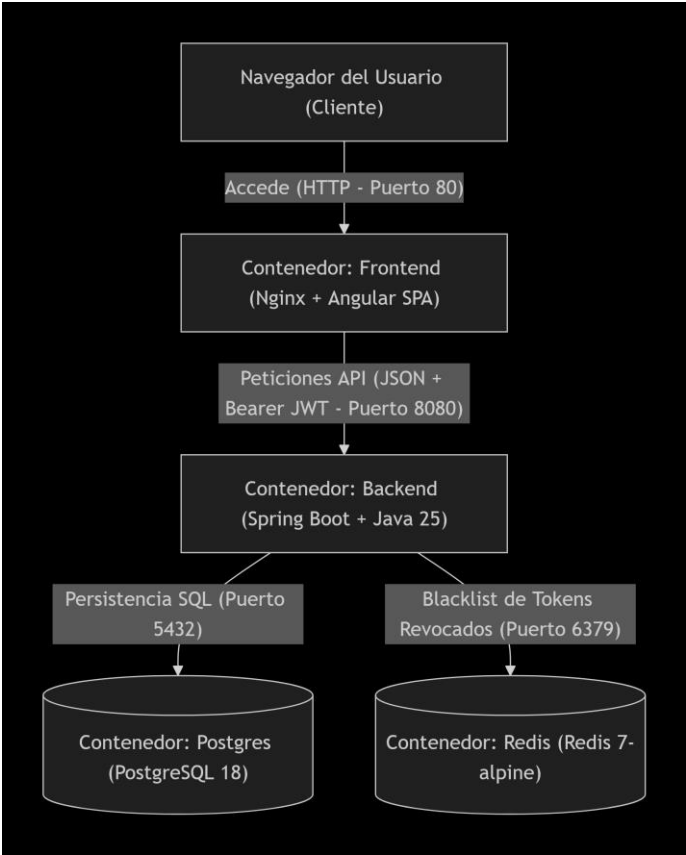


Figura 1. Diagrama C4 Nivel 2 — Contenedores de Software Artisync.

Contenedor	Tecnología	Protocolo	Puerto	Descripción
Backend	Spring Boot 4.1.0 / Java	HTTP/REST	8080	API REST con Spring Security 6 y JWT
Frontend	Angular 22 / Nginx	HTTP	80	SPA que consume la API y sirve la UI
Base de Datos	PostgreSQL 18	TCP	5432	Motor relacional; esquema gestionado por Flyway
Caché / Blacklist	Redis 7	TCP	6379	Lista negra de JTI y caché de sesiones

4.2 Diagrama de Clases UML (Backend Completo)

[INSERTAR IMAGEN: Diagrama de Clases UML — Backend Spring Boot (PNG, alta resolución)]

4.3 Tabla de Correspondencia Java ↔ PostgreSQL

Tabla: usuarios

Atributo Java	Tipo Java	Tipo PostgreSQL	Restricciones y Justificación
---------------	-----------	-----------------	-------------------------------

id	Long	BIGSERIAL	PK; secuencia PostgreSQL;
nombre	String	VARCHAR(100)	NOT NULL; nombre del usuario
email	String	VARCHAR(255)	UNIQUE NOT NULL; índice CREATE UNIQUE INDEX idx_usuarios_email
passwordHash	String	VARCHAR(255)	BCrypt costo 12; nunca serializado (@JsonIgnore)
rol	enum Rol	VARCHAR(20)	CHECK: ROLE_USER, ROLE_ADMIN; no ENUM nativo PostgreSQL
creadoEn	Instant	TIMESTAMPTZ	UTC; gestionado por @CreationTimestamp
actualizadoEn	Instant	TIMESTAMPTZ	UTC; gestionado por set_actualizado_en()
activo	boolean	BOOLEAN	Soft delete; DEFAULT TRUE; false = eliminado lógicamente

4.4 Diagrama de Secuencia: Flujo de Autenticación JWT

[INSERTAR IMAGEN: Diagrama de Secuencia — Flujo Login JWT con 11 pasos (PNG)]

Los 11 pasos del flujo de autenticación JWT son:

1. Cliente Angular envía POST /api/auth/login con {email, password}.
2. AuthController delega a AuthenticationManager.authenticate().
3. Spring Security consulta UserDetailsService.loadUserByUsername().
4. UserDetailsService consulta UsuarioRepository.findByEmail().
5. PostgreSQL devuelve el usuario; Hibernate lo mapea a la entidad.
6. Spring Security verifica el hash BCrypt con PasswordEncoder.matches().
7. JwtService.generateToken() firma el JWT con la clave secreta (HS256).
8. El servidor devuelve {accessToken, refreshToken, expiresIn}.
9. Angular guarda el accessToken en memoria (NO en localStorage) y el refreshToken en cookie HttpOnly.
10. En cada solicitud posterior, el JwtInterceptor agrega Authorization: Bearer [token].
11. JwtAuthFilter valida el token, consulta Redis (blacklist por JTI) y establece el SecurityContext.

4.5 Diagrama Entidad-Relación (DER)

[INSERTAR IMAGEN: Diagrama Entidad-Relación generado desde pgAdmin 4 o DBeaver (PNG)]

4.6 Script Flyway de Migración Inicial

El script `V1_____schema_inicial.sql` se encuentra en `Backend/src/main/resources/db/migration/`. Debe ejecutarse sin errores en PostgreSQL 18 con `ddl-auto: validate`.

[INSERTAR o referenciar:

`Backend/src/main/resources/db/migration/V1__schema_inicial.sql]`

4.7 Diccionario de Datos

Tabla: usuarios

Columna	Tipo PostgreSQL	Nulo	Valor por Defecto	Descripción del Negocio
id	BIGSERIAL	NO	NEXTVAL	Identificador único del usuario
nombre	VARCHAR(100)	NO	---	Nombre completo del usuario
email	VARCHAR(255)	NO	---	Correo electrónico; único; usado como credencial de login
password_hash	VARCHAR(255)	NO	---	Hash BCrypt costo 12; nunca expuesto en respuestas JSON
rol	VARCHAR(20)	NO	'ROLE_USER'	Rol de autorización; CHECK IN ('ROLE_USER','ROLE_ADMIN')
activo	BOOLEAN	NO	TRUE	Soft delete: FALSE indica usuario eliminado lógicamente
creado_en	TIMESTAMPTZ	NO	NOW()	Timestamp UTC de creación
actualizado_en	TIMESTAMPTZ	NO	NOW()	Timestamp UTC de última actualización

Tabla: servicios

Columna	Tipo PostgreSQL	Nulo	Valor por Defecto	Descripción del Negocio
id_servicio	BIGSERIAL	NO	NEXTVAL	Identificador único del servicio

id_perfil	INT	NO	---	FK al perfil del creador que ofrece el servicio
id_subcategoria	INT	NO	---	FK a la subcategoría del servicio
titulo_servicio	VARCHAR(150)	NO	---	Título del servicio ofrecido
descripcion_detallada	TEXT	NO	---	Descripción detallada del servicio
precio_base	DECIMAL(10,2)	NO	---	Precio base del servicio
url_miniatura	VARCHAR(255)	SI	---	URL de la imagen miniatura del servicio

5. Implementación

5.1 Estructura del Repositorio (Monorepo)

Ruta	Contenido
Backend/	Código fuente y configuración del backend basado en Spring Boot
Backend/src/main/java/uteq/edu/ec/artisync/	Código fuente Java organizado en paquetes por capa (entity, dto, repository, service, controller, security, config)
Backend/src/main/resources/application.properties	Archivo de propiedades de Spring Boot
Backend/src/main/resources/db/migration/	Migraciones SQL gestionadas por Flyway para la base de datos PostgreSQL
Backend/pom.xml	Descriptor de dependencias de Maven para el Backend
Backend/Dockerfile	Dockerfile multietapa para compilar y ejecutar el servicio Backend con Java 25
Frontend/	Código fuente y configuración del cliente web basado en Angular 22
Frontend/src/app/	Estructura interna de Angular (componentes, enrutamiento, estilos, layouts)
Frontend/package.json	Definición de dependencias npm y scripts del Frontend
Frontend/nginx.conf	Configuración de Nginx para servir la aplicación Angular en modo SPA
Frontend/Dockerfile	Dockerfile multietapa que compila el frontend en Node y lo sirve bajo Nginx

database/	Recursos compartidos e independientes para la base de datos
docs/	Documentación del proyecto (ADRs, diagramas de diseño, diccionario de datos)
docs/adr/ADR-003-jwt-redis.md	Decisión de diseño para el uso de Redis en la blacklist de JWT
docs/diccionario_datos.md	Diccionario de datos completo del esquema PostgreSQL
docker-compose.yml	Orquestador de Docker para levantar de manera unificada todos los servicios
.env.example	Ejemplo de configuración de las variables de entorno necesarias para la infraestructura local
README.md	Guía de introducción rápida y comandos esenciales para ejecutar la aplicación

5.2 Configuración de Spring Boot (application.properties)

[INSERTAR o referenciar: Backend/src/main/resources/application.properties comentado]

5.3 Implementación del Filtro JWT (JwtAuthFilter)

El filtro JwtAuthFilter extiende OncePerRequestFilter e intercepta cada solicitud HTTP para:

1. Extraer el token del encabezado Authorization: Bearer [token].
2. Validar la firma y la expiración con JwtService.validateToken().
3. Consultar Redis para verificar que el JTI no está en la blacklist.
4. Establecer el UsernamePasswordAuthenticationToken en el SecurityContextHolder.

[INSERTAR código completo de JwtAuthFilter.java o enlace al archivo en el repositorio]

5.4 Tabla de Endpoints Implementados

Método	URL	Auth	Descripción	Respuesta
POST	/api/auth/registro	No	Registrar nuevo usuario; password hashado con BCrypt	201 Created
POST	/api/auth/login	No	Autenticar; devuelve accessToken y refreshToken	200 OK + JWT
POST	/api/auth/logout	JWT	Agrega el JTI a la blacklist Redis con TTL	204 No Content
POST	/api/auth/refresh	RefreshToken	Emite un nuevo accessToken sin re-autenticar	200 OK + JWT
GET	/api/usuarios/me	JWT	Devuelve el perfil del usuario autenticado	200 OK
GET	/api/servicios	JWT	Listar servicios con paginación: ?page=0&size=10&sort=id,asc	200 OK + Page
GET	/api/servicios/{id}	JWT	Buscar un servicio por ID; 404 si no existe	200 OK / 404
POST	/api/servicios	JWT	Crear un nuevo servicio; valida con @Valid	201 Created
PUT	/api/servicios/{id}	JWT	Actualizar un servicio completo	200 OK
DELETE	/api/servicios/{id}	JWT	Soft delete (activo=false)	204 No Content
GET	/api/docs	No	Swagger UI generado por Springdoc OpenAPI 2.8.8	200 OK (HTML)

5.5 Estructura del JWT Emitido

Claim	Valor de Ejemplo	Descripción
sub	"1"	ID del usuario (sujeto del token)
email	"juan@uteq.edu.ec"	Email del usuario autenticado
rol	"ROLE_USER"	Rol para autorización en Spring Security
iat	1735689600	<i>Issued At</i> - Timestamp Unix de emisión
exp	1735693200	<i>Expiration Time</i> - iat + 3600 s (1 hora)

[INSERTAR: payload real decodificado del JWT emitido por el sistema (captura de jwt.io o similar)]

5.6 Implementación Angular: Interceptor JWT y AuthGuard

[INSERTAR código de `auth.service.ts`, `jwt.interceptor.ts` y `auth.guard.ts`]

6. Pruebas

6.1 Pruebas Unitarias JUnit 5

#	Caso de Prueba	Método de Prueba	Esperado	Resultado
1	Login con credenciales correctas	loginExitoso()	200 OK + JWT	[PASS/FAIL]
2	Login con contraseña incorrecta	loginClaveIncorrecta()	401 Unauthorized	[PASS/FAIL]
3	Registro con email ya existente	registroEmailDuplicado()	409 Conflict	[PASS/FAIL]
4	Acceso a endpoint protegido sin token	accesoSinToken()	401 Unauthorized	[PASS/FAIL]
5	Acceso a endpoint protegido con token válido	accesoConTokenValido()	200 OK	[PASS/FAIL]
6	CRUD: crear servicio	crearServicioExitoso()	201 Created	[PASS/FAIL]
7	CRUD: listar servicios con paginación	listarServiciosPaginado()	200 OK + Page	[PASS/FAIL]
8	CRUD: actualizar servicio	actualizarServicioExitoso()	200 OK	[PASS/FAIL]
9	CRUD: soft delete servicio	eliminarServicioSoftDelete()	204 No Content	[PASS/FAIL]

[INSERTAR captura de pantalla: salida de mvn test con todos los tests en VERDE]

6.2 Pruebas de Integración con Postman

#	Escenario	Endpoint	Evidencia
1	Registro de usuario con datos válidos	POST /api/auth/registro	[Captura Postman]
2	Login exitoso con JWT visible en respuesta	POST /api/auth/login	[Captura Postman]
3	Acceso protegido con Bearer token	GET /api/usuarios/me	[Captura Postman]
4	Logout: JTI invalidado en Redis	POST /api/auth/logout	[Captura Postman]
5	Reuso de token revocado → 401	GET /api/servicios	[Captura Postman]

6	CRUD: POST crear servicio	POST /api/servicios	[Captura Postman]
7	CRUD: GET lista paginada de servicios	GET /api/servicios?page=0&size=5	[Captura Postman]
8	CRUD: GET servicio por ID	GET /api/servicios/{id}	[Captura Postman]
9	CRUD: PUT actualizar servicio	PUT /api/servicios/{id}	[Captura Postman]
10	CRUD: DELETE soft delete servicio	DELETE /api/servicios/{id}	[Captura Postman]
11	Swagger UI con endpoints documentados	GET /api/docs	[Captura Postman/Browser]

6.3 Métricas de Rendimiento

Endpoint	Tiempo Promedio	Tiempo P95	Observación
POST /api/auth/login	[X] ms	[X] ms	Incluye BCrypt verify (costo 12)
GET /api/servicios (sin caché)	[X] ms	[X] ms	Sin Redis cache
GET /api/servicios (con Redis)	[X] ms	[X] ms	Comparar con fila anterior
POST /api/servicios	[X] ms	[X] ms	Con validación @Valid y commit PostgreSQL

7. Seguridad Implementada

Control	Implementación	OWASP Top 10 2021	Evidencia
Hash de contraseñas	BCryptPasswordEncoder costo 12 en registro y login	A02 - Fallas criptográficas	Captura BD: hash \$2a\$12\$. . .
Tokens JWT de corta vida	accessToken: 1 h; refreshToken: 7 d; claims estándar RFC 7519	A07 - Fallas de autenticación	Payload JWT decodificado (sección 5.5)
Blacklist de tokens	JWT revocado en Redis con TTL = tiempo restante del token	A07 - Fallas de autenticación	Prueba: logout + reuso → 401 (sección 6.2)
Cabeceras de seguridad HTTP	X-Content-Type-Options, X-Frame-Options, CSP via Spring Security	A05 - Configuración incorrecta	curl -I al endpoint: cabeceras visibles
CORS configurado	Lista blanca de orígenes en SecurityConfig; allowCredentials	A05 - Configuración incorrecta	Prueba desde dominio no permitido → 403
Sin SQL directo	Spring Data JPA + @Query parametrizado; cero concatenación SQL	A03 - Inyección SQL	Revisión de código: ningún String sql = "... " + variable
Roles de acceso	ROLE_USER y ROLE_ADMIN con @PreAuthorize en endpoints sensibles	A01 - Control de acceso roto	Prueba de acceso denegado a ruta ADMIN con token USER

8. Docker Compose y Despliegue

8.1 Archivo docker-compose.yml

[INSERTAR: docker-compose.yml completo con 4 servicios + healthchecks + depends_on]

8.2 Instrucciones de Ejecución (README.md)

1. `git clone https://github.com/JohanCarvajal04/Proyecto-WEB-ARTISYNC.git && git checkout entrega-1b`
2. `cp .env.example .env` y editar el archivo `.env` con las credenciales del entorno local
3. `docker compose up --build -d`
4. `docker compose ps`
5. Acceder a la aplicación: **Frontend** `http://localhost:80` | **Swagger UI** `http://localhost:8080/api/swagger-ui.html` | **Actuator** `http://localhost:8080/actuator/health`

Servicio	Imagen Base	Puerto	Health Check	Depends On
postgres	postgres:18	5432	pg_isready -U pfc_user	---
redis	redis:7-alpine	6379	redis-cli ping	---
backend	eclipse-temurin:25-jre-alpine	8080	GET /actuator/health	postgres, redis
frontend	nginx:alpine	80	curl http://localhost	backend

9. Historial de Git

[INSERTAR captura: `git log --oneline --graph --all` mostrando commits de TODOS los integrantes]

[INSERTAR captura o confirmar: `tag v0.1.0-entrega-1b` existe y apunta al commit de entrega]

Convenciones de Conventional Commits utilizadas en este repositorio:

Prefijo	Uso	Ejemplo
feat:	Nueva funcionalidad	feat: implementar JwtAuthFilter con blacklist Redis
fix:	Corrección de bug	fix: corregir validación de expiración en JwtService
test:	Agregar o modificar pruebas	test: añadir casos JUnit5 para AuthController
docs:	Documentación	docs: agregar ADR-003-jwt-redis.md
refactor:	Refactorización sin cambio de funcionalidad	refactor: extraer lógica de claims a JwtService
chore:	Tareas de mantenimiento	chore: actualizar docker-compose.yml con healthchecks

10. Conclusiones

Conclusión Objetivo 1: Módulo de Autenticación JWT

[Párrafo de 150—200 palabras: ¿Se logró implementar el módulo de autenticación?

¿Cuál fue el mayor desafío técnico (ej.: configuración de OncePerRequestFilter, integración Redis)?

¿Cómo se resolvió?]

Conclusión Objetivo 2: Gestión de Tokens y Blacklist Redis

[Párrafo de 150—200 palabras: ¿Se logró la gestión de refresh tokens y la blacklist JTI en Redis? ¿Cuál fue el mayor desafío técnico? ¿Cómo se resolvió?]

Conclusión Objetivo 3: CRUD con Spring Data JPA y Flyway

[Párrafo de 150—200 palabras: ¿Se logró el CRUD completo con paginación y Flyway? ¿Cuál fue el mayor desafío (ej.: LazyInitializationException, ddl-auto: validate)? ¿Cómo se resolvió?]

Conclusión Objetivo 4: Docker Compose y Despliegue

[Párrafo de 150—200 palabras: ¿Se logró el despliegue reproducible con docker compose up? ¿Cuál fue el mayor desafío técnico? ¿Cómo se resolvió?]

Proyección a Entrega 2 (Semana 12)

En la Entrega 2, se planea implementar el módulo de flujo de pedidos y el motor de flujos de trabajo (pedidos, historial_estados_pedido), el sistema de mensajería en tiempo real vía WebSockets (salas_chat, mensajes), y la integración inicial con la API de PayPal para el manejo de pagos en garantía (Escrow) a través de la entidad pagos_garantia.

11. Referencias Bibliográficas

Walls, C. (2022). *Spring Boot in action* (2da ed.). Manning Publications. Spilca, L. (2024). *Spring Security in action* (2da ed.). Manning Publications.

Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). IETF. <https://tools.ietf.org/html/rfc7519>

OWASP Foundation. (2021). *OWASP Top 10: 2021*. <https://owasp.org/Top10/> Fielding, R. T. (2000).

Architectural styles and the design of network-based software architectures [Tesis doctoral, UC Irvine].

PostgreSQL Global Development Group. (2024). *PostgreSQL 18 documentation*. <https://www.postgresql.org/docs/18/>

Spring Framework Team. (2024). *Spring Security reference documentation*. <https://docs.spring.io/spring-security/reference/>

[AGREGAR: artículo indexado IEEE/ACM/Scopus sobre seguridad JWT en aplicaciones web]

Anexo A — Lista de Verificación Final

Bloqueantes (verificar primero)

Ítem	Estado
☐ <code>docker compose up</code> levanta TODO sin intervención manual	☐ OK ☐ Pendiente
☐ Ninguna contraseña en texto plano en la BD (debe verse <code>\$2a\$12\$...</code>)	☐ OK ☐ Pendiente
☐ <code>accessToken</code> NO está en <code>localStorage</code> (debe estar en memoria Angular)	☐ OK ☐ Pendiente
☐ Cero SQL concatenado en el código Java	☐ OK ☐ Pendiente

Informe Técnico

Ítem	Estado
☐ Portada con integrantes, cédulas, URL del repositorio y rama entrega-1b	☐ OK ☐ Pendiente
☐ Resumen ejecutivo con métrica numérica real (tiempo, cobertura, etc.)	☐ OK ☐ Pendiente
☐ Marco teórico con mínimo 8 referencias APA 7; al menos 3 indexadas	☐ OK ☐ Pendiente
☐ Diagrama C4 Nivel 2 con backend, frontend, PostgreSQL y Redis	☐ OK ☐ Pendiente
☐ Diagrama de secuencia JWT con los 11 pasos completos	☐ OK ☐ Pendiente
☐ Script <code>V1__schema_inicial.sql</code> ejecutable en PostgreSQL 18 sin errores	☐ OK ☐ Pendiente
☐ Tabla de endpoints con ≥ 10 endpoints documentados	☐ OK ☐ Pendiente
☐ Tabla de seguridad con ≥ 6 controles OWASP y evidencia	☐ OK ☐ Pendiente

Evidencias y Pruebas

Ítem	Estado
☐ Capturas Postman: login exitoso con JWT visible en respuesta	☐ OK ☐ Pendiente
☐ Capturas Postman: CRUD completo (POST / GET lista paginada / GET id / PUT / DELETE)	☐ OK ☐ Pendiente
☐ Captura: <code>mvn test</code> con ≥ 5 pruebas en VERDE	☐ OK ☐ Pendiente

☐ Captura: Swagger UI con todos los endpoints documentados	[] OK [] <i>Pendiente</i>
☐ Tabla de métricas con tiempos de respuesta reales medidos	[] OK [] <i>Pendiente</i>

Repositorio Git y Entrega

Ítem	Estado
☐ docker-compose.yml con 4 servicios + healthchecks	[] OK [] <i>Pendiente</i>
☐ README.md con instrucciones de ejecución en 5 pasos ejecutables	[] OK [] <i>Pendiente</i>
☐ Historial Git con commits de TODOS los integrantes con Conventional Commits	[] OK [] <i>Pendiente</i>
☐ Tag v0.1.0-entrega-1b existe y apunta al commit de entrega	[] OK [] <i>Pendiente</i>
☐ PDF nombrado PFC_Entrega1B_ARTISYNC.pdf	[] OK [] <i>Pendiente</i>
☐ PDF adjuntado en la Tarea del SGA antes del cierre de la Semana 9	[] OK [] <i>Pendiente</i>